计算机组成与结构
—指令系统2

李瑞　　教授

蒋志平　副教授

计算机科学与技术学院

温故 —关于指令系统

- 虚拟存储系统有哪些功能?
- 虚拟存储的工作原理?
- 简述段式虚拟存储转换的原理和优缺点
- 简述页式虚拟存储转换的原理和优缺点
- 简述有哪些优化虚拟存储性能的方法?
- CPU执行指令的4个步骤是?
- 指令的格式是什么?

本节学习要点

- 指令格式
- 可执行程序的结构
- 对Emu8086做简单使用介绍
- CPU是如何定位程序的? (寄存器）
- CPU是如何定位程序中的代码或数据的? （寻址方式）

指令格式

指令格式

高级语言

```
auto area() -> float {
    return a * b;
}
```

编译

汇编语言

```
Load r1, a
Load r2, b
MUL r1, r2
RET
```

直译

机器代码

```
00010101000 01110
10101000 01011101
1010101011...
```

MUL r1, r2被转译为

这是瞎编的~

“01110101010000 1011”

这一条指令包含以下4条信息：

- MUL指令的编号
- MUL指令能支持两个操作数
- 操作数1指向r1寄存器
- 操作数2指向r2寄存器

指令格式

- 指令 = 操作码Opcode + 操作数 Operand (0~N)
- 操作数，也叫地址码

地址码的内容可以是：地址、数值

地址码的目标可以是：

源/目的操作数，结果，下一条指令

地址（顺序/跳转）

例如，某定长指令格式



指令格式

- 常用的地址码数目：
  - 三地址：ADD Y, A, B ; Y=A+B
  - 双地址：ADD Y, A ; Y=Y+A
  - 单地址：ADD C ; AC=AC+C
  - 零地址

指令格式

例如：计算一个典型的算术表达式：

$$x = \frac{a \times b + c - d}{e + f}$$

用三地址指令编写的程序如下：

```
MUL   X, A, B      ;X←(A)×(B)
ADD   X, X, C      ;X←(X)＋(C)
SUB   X, X, D      ;分子的计算结果在X中
ADD   Y, E, F      ;计算分母,存入Y
DIV   X, X, Y      ;最后结果在X单元中
```

指令格式

$$x = \frac{a \times b + c - d}{e + f}$$

用普通二地址指令编写的程序：

```
MOVE  X, A        ;复制临时变量到X中
MUL   X, B
ADD   X, C
SUB   X, D        ;X中存放分子运算结果
MOVE  Y, E        ;复制临时变量到Y中
ADD   Y, F        ;Y中存放分母运算结果
DIV   X, Y        ;最后结果在X单元中
```

指令格式

$$x = \frac{a \times b + c - d}{e + f}$$

用多寄存器结构的二地址指令编写程序：

```
MOVE  R1, A     ;操作数a取到寄存器R1中
MUL   R1, B
ADD   R1, C
SUB   R1, D     ;R1中存放分子运算结果
MOVE  R2, E
ADD   R2, F     ;R2中存放分母运算结果
DIV   R1, R2    ;最后结果在R1中
MOVE  X, R1     ;最后结果存入X中
```

指令格式

$$x = \frac{a \times b + c - d}{e + f}$$

用一地址指令编写的程序：

```
LOAD    E       ;先计算分母,
                ;取一个操作数到累加器中
ADD     F       ;分母运算结果在累加器中
STORE   X       ;保存分母运算结果到X中
LOAD    A       ;开始计算分子
MUL     B
ADD     C
SUB     D       ;累加器中是分子运算结果
DIV     X       ;最后运算结果在累加器中
STORE   X       ;保存最后运算结果到X中
```

指令格式

用0地址指令编写程序：

```
PUSH    A       ;操作数a压入堆栈
PUSH    B       ;操作数b压入堆栈
MUL             ;栈顶两数相乘，结果压回堆顶
PUSH    C
ADD
PUSH    D
SUB             ;栈顶是分子运算的结果
PUSH    E
PUSH    F
ADD
DIV             ;栈顶是最后运算的结果
POP     X       ;保存最后运算结果
```

$$x = \frac{a \times b + c - d}{e + f}$$

ab*c+d-ef+/

指令格式

不同地址个数指令的特点及适用场合

地址数目	程序的长度	程序存储量	程序执行速度	适用场合
三地址	最短	最大	一般	向量，矩阵运算为主
二地址	较短	很大	很低	一般不宜采用
一地址	较长	较大	较快	连续运算，硬件结构简单
零地址	最长	最小	最低	嵌套，递归，变量较多

指令格式

关于地址码个数结论：

- 对于一般商用处理机，采用多寄存器结构的二地址指令是最理想的。
- 如果强调硬件结构简单，并且以连续运算（如求累加和等）为主，宜采用一地址结构。
- 对于以向量、矩阵运算为主的处理机，最好采用三地址结构。
- 部分RISC处理机也采用三地址指令。
- 对于解决递归问题为主的处理机，宜采用零地址结构。编程容易、节省程序存储量。

指令编码

指令编码

- 指令操作码(Op code)编码方式:
- 编码的考量因素
- 编码方法
  - 定长编码
  - 不定长编码
    - 霍夫曼Huffman Coding
    - 扩展操作码

指令编码

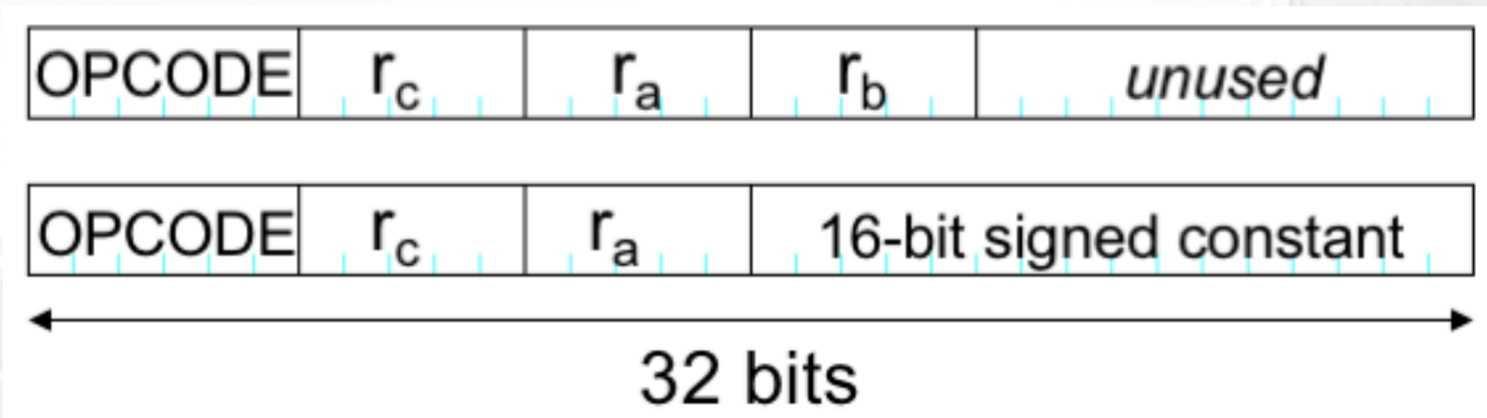
构成一条指令的二进制的位数称为指令长度，一般都是字节的整数倍。

- 内存大小与组织
- CPU数据总线宽度
- CPU内部寄存器的数量
- 寻址方式
- 指令数量

指令编码

定长指令编码：

- 所有指令的编码Opcode长度相同！
  - 不管几操作数的指令，opcode都一样长！
- 显然，会有指令bit浪费，或指令容量受限



指令编码

Huffman指令编码：

- **让高频指令的opcode更短！（不care操作数长度！）**

指令编码

例：假设一台模型计算机共有7种不同的操作码，已知各种操作码在程序中出现的概率如下表，利用固定长度编码法进行操作码编码。

指令	I_1	I_2	I_3	I_4	I_5	I_6	I_7
概率	0.45	0.30	0.15	0.05	0.03	0.01	0.01

解：由于N=7 因此，指令操作码固定长度为

$$[lbN]=[lb7]=3$$

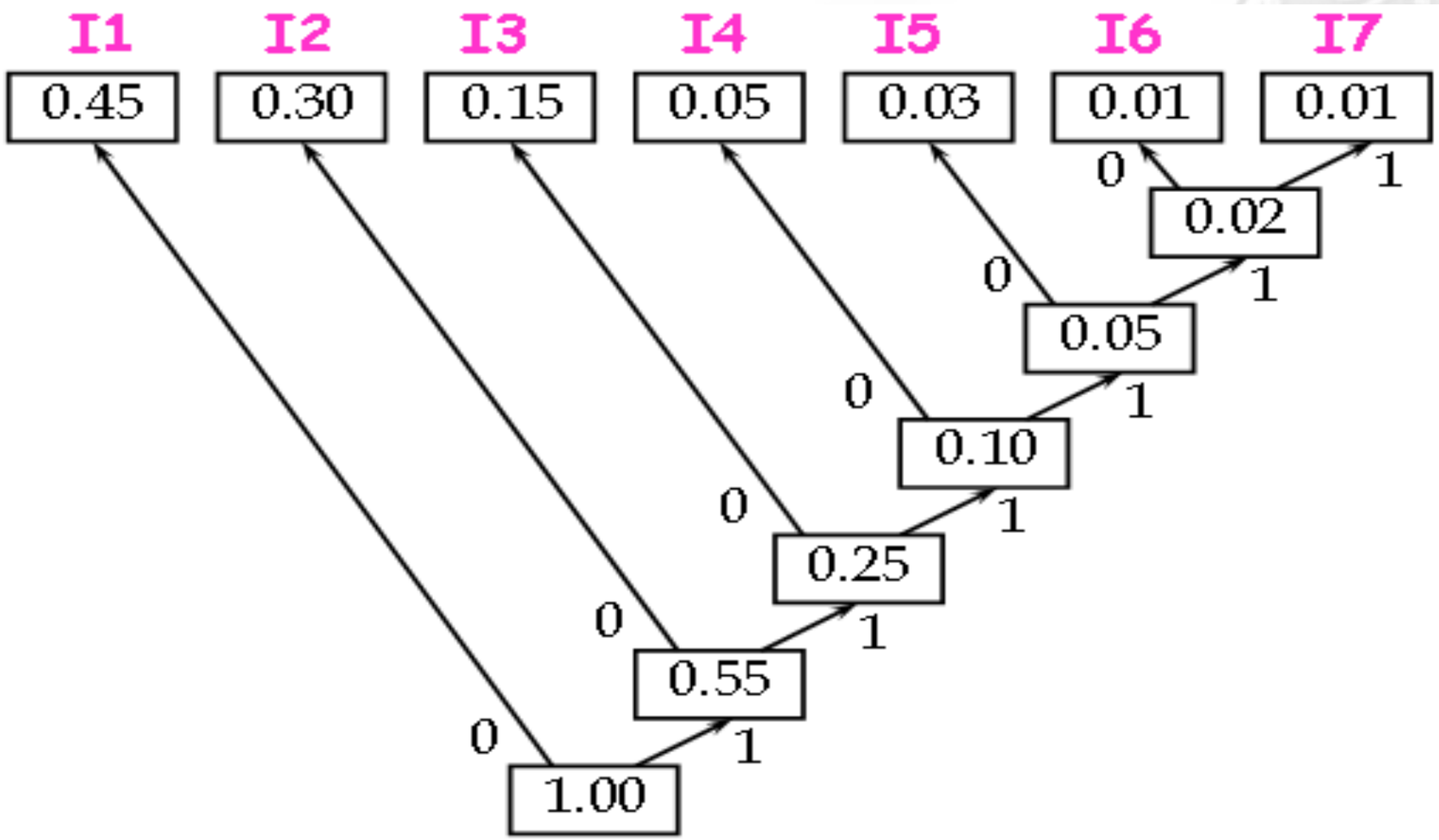
编码结果：

指令序号	概率	编码	操作码长度
I_1	0.45	000	3位
I_2	0.30	001	3位
I_3	0.15	010	3位
I_4	0.05	011	3位
I_5	0.03	100	3位
I_6	0.01	101	3位
I_7	0.01	110	3位

指令编码

例：利用Huffman 编码法进行操作码编码。

指令	I_1	I_2	I_3	I_4	I_5	I_6	I_7
概率	0.45	0.30	0.15	0.05	0.03	0.01	0.01



Huffman编码树生成过程

指令编码

编码结果:

指令序号	概率	Huffman编码法	操作码长度
I_1	0.45	0	1位
I_2	0.30	10	2位
I_3	0.15	110	3位
I_4	0.05	1110	4位
I_5	0.03	11110	5位
I_6	0.01	111110	6位
I_7	0.01	111111	6位

采用Huffman编码法的操作码平均长度:

$0.45 \times 1 + 0.30 \times 2 + 0.15 \times 3 + 0.05 \times 4 + 0.03 \times 5 + 0.01 \times 6 + 0.01 \times 6$

$= 1.97$（位）

指令编码

Huffman操作码的主要缺点：

1) 操作码长度很不规整，硬件译码困难
2) 与地址码共同组成固定长的指令比较困难

解决方法之一：　指令操作码的扩展技术

指令编码

扩展指令编码：

- **让操作数多的指令opcode更短，**
- **操作数少的指令opcode最长**

指令编码

原则：短码不能是长码的前缀。

扩展指令编码

15-15-15 编码方法

15
- 0000
- 0001
- ……
- 1110

15
- 1111 0000
- 1111 0001
- ……
- 1111 1110

15
- 1111 1111 0000
- 1111 1111 0001
- ……
- 1111 1111 1110

图5.4

8-64-512 编码方法

8
- 0000
- 0001
- ……
- 0111

64
- 1000 0000
- 1000 0001
- ……
- 1111 0111

512
- 1000 1000 0000
- 1000 1000 0001
- ……
- 1111 1111 0111

指令编码

- 短的操作码与多种地址码配合
- 长的操作码与简单地址码组合
- 指令长度一般设计为总线宽度的整数倍
- 指令的长度为最小可寻址单位的整数倍
- 影响寻址位的因素：
  - 寻址方式的个数
  - 操作数数目
  - 寄存器数目
  - 地址范围
  - 地址可寻址最小单元

指令编码

例：假设某型号的计算机共有14条指令，各条指令的使用频率分别为：0.01、0.15、0.12、0.03、0.02、0.04、0.02、0.04、0.01、0.13、0.15、0.14、0.11、0.03。

试给出定长、只能有两种码长的扩展操作码的两种编码方案，并计算各种方案的平均码长。

【解】

指令	I_1	I_2	I_3	I_4	I_5	I_6	I_7
概率	0.15	0.15	0.14	0.13	0.12	0.11	0.04
指令	I_8	I_9	I_{10}	I_{11}	I_{12}	I_{13}	I_{14}
概率	0.04	0.03	0.03	0.02	0.02	0.01	0.01

指令编码

定长编码：

指令	I_1	I_2	I_3	I_4	I_5	I_6	I_7
概率	0.15	0.15	0.14	0.13	0.12	0.11	0.04
扩展编码	0000	0001	0010	0011	0100	0101	0110
码长	4	4	4	4	4	4	4
指令	I_8	I_9	I_{10}	I_{11}	I_{12}	I_{13}	I_{14}
概率	0.04	0.03	0.03	0.02	0.02	0.01	0.01
扩展编码	0111	1000	1001	1010	1011	1100	1101
码长	4	4	4	4	4	4	4

其操作码的平均码长：**H＝4**（位）。

指令编码

两种码长的扩展操作码：（方法一）

指令	I_1	I_2	I_3	I_4	I_5	I_6	I_7
概率	0.15	0.15	0.14	0.13	0.12	0.11	0.04
扩展编码	000	001	010	011	100	101	110
码长	3	3	3	3	3	3	3
指令	I_8	I_9	I_{10}	I_{11}	I_{12}	I_{13}	I_{14}
概率	0.04	0.03	0.03	0.02	0.02	0.01	0.01
扩展编码	111000	111001	111010	111011	111100	111101	111110
码长	6	6	6	6	6	6	6

其操作码平均码长为：$H=\sum_{i=1}^{n} p_i l_i$

$$H=(0.15+0.15+0.14+0.13+0.12+0.11+0.04)\times 3+$$
$$(0.04+0.03+0.03+0.02+0.02+0.01+0.01)\times 6$$
$$=0.84\times 3+0.16\times 6=3.48\ (\text{位})$$

指令编码

大概率

指令	I_1	I_2	I_3	I_4	I_5	I_6	I_7
概率	0.15	0.15	0.14	0.13	0.12	0.11	0.04
扩展编码							
码长							
指令	I_8	I_9	I_{10}	I_{11}	I_{12}	I_{13}	I_{14}
概率	0.04	0.03	0.03	0.02	0.02	0.01	0.01
扩展编码							
码长							

小概率

指令编码

大概率

两种码长的扩展操作码：（方法二）

指令	I_1	I_2	I_3	I_4	I_5	I_6	I_7
概率	0.15	0.15	0.14	0.13	0.12	0.11	0.04
扩展编	000	001	010	011	100	101	11000
码长	3	3	3	3	3	3	5
指令	I_8	I_9	I_{10}	I_{11}	I_{12}	I_{13}	I_{14}
概率	0.04	0.03	0.03	0.02	0.02	0.01	0.01
扩展编	11001	11010	11011	11100	11101	11110	11111
码长	5	5	5	5	5	5	5

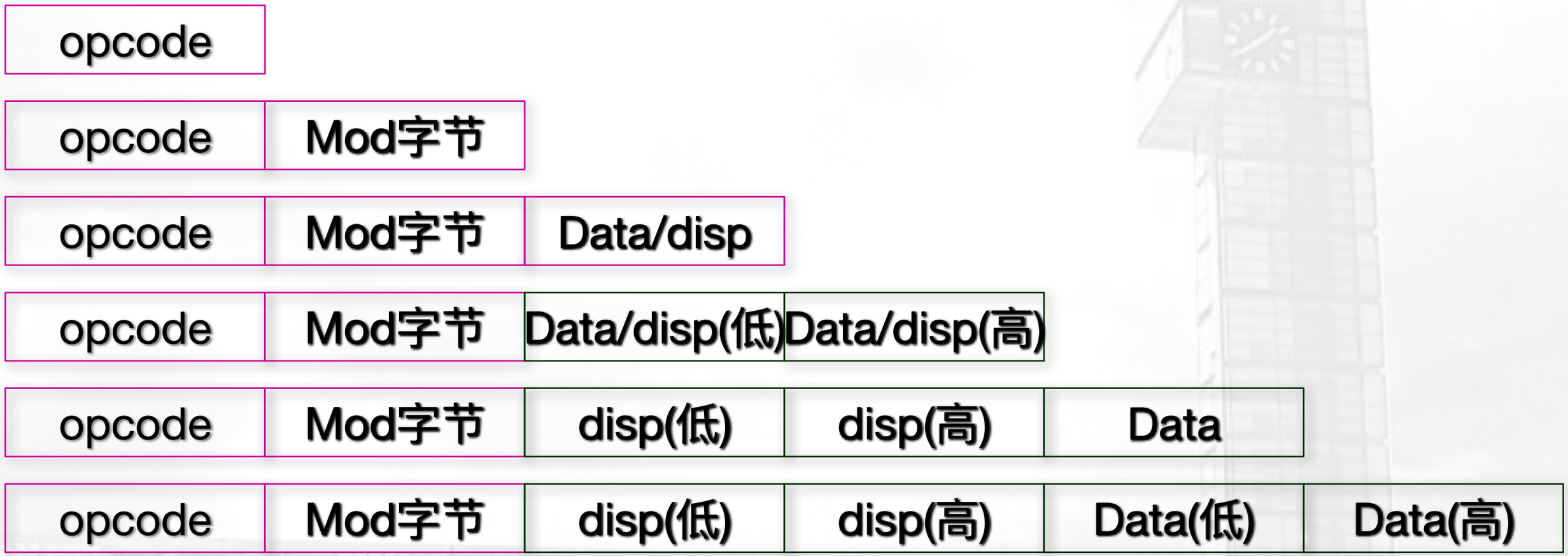
小概率

其操作码平均码长为：$H=\sum_{i=1}^{n} p_i l_i$

$$H=(0.15+0.15+0.14+0.13+0.12+0.11)\times 3+$$
$$(0.04+0.04+0.03+0.03+0.02+0.02+0.01+0.01)\times 5$$
$$=0.8\times 3+0.2\times 5=3.4\ \text{（位）}$$

指令格式

8086 / 8088指令编码由1～6个字节组成，它包括操作码（第一字节）、寻址方式（第二字节）和操作数（第三到第六字节）三部分组成p



可执行程序的两个结构

可执行程序的两个结构

- 一个最简单的C代码：　　其实一点也不简单...

```
int main() {
    printf("Hello world!\n");
}
```

可执行程序的两个结构

- 一个最简单的C代码：　　其实一点也不简单...

```
int main() {
    printf("Hello world!\n");
}
```

如何定义为程序的入口?

调用libc库中的printf函数

如何调用这个函数？链接它！

动态链接？通过OS找到libc

这是一个常量，存哪?

静态链接，把libc的printf及其依赖都打包进来

如何调用printf方法?

数据和代码可以流水帐一样堆起来么？ No!

可执行程序中上怎么存另一块代码?

可执行程序是有结构的

可执行程序的两个结构

- 一个最简单的C代码：

```
int main() {
    printf("Hello world!\n");
}
```

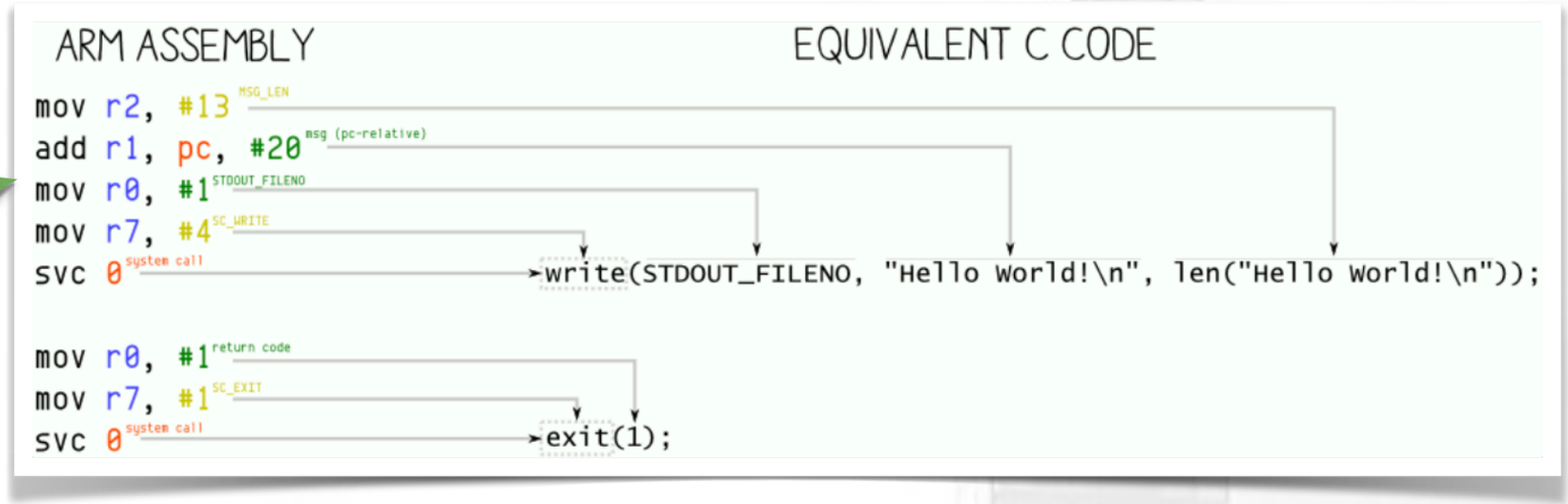
经历2个步骤变成可执行程序：

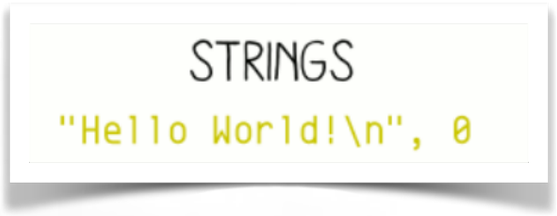
- 编译（把自己的程序逻辑变为机器代码）
- 链接（链接其它库的函数调用）

- 可执行程序的静态结构（ELF为例）
  - ELF Header
  - Sections
    - CODE
    - DATA
    - BSS
    - RODATA
    - HEAP
    - STACK
    - …
  - Section Header Table

可执行程序的两个结构

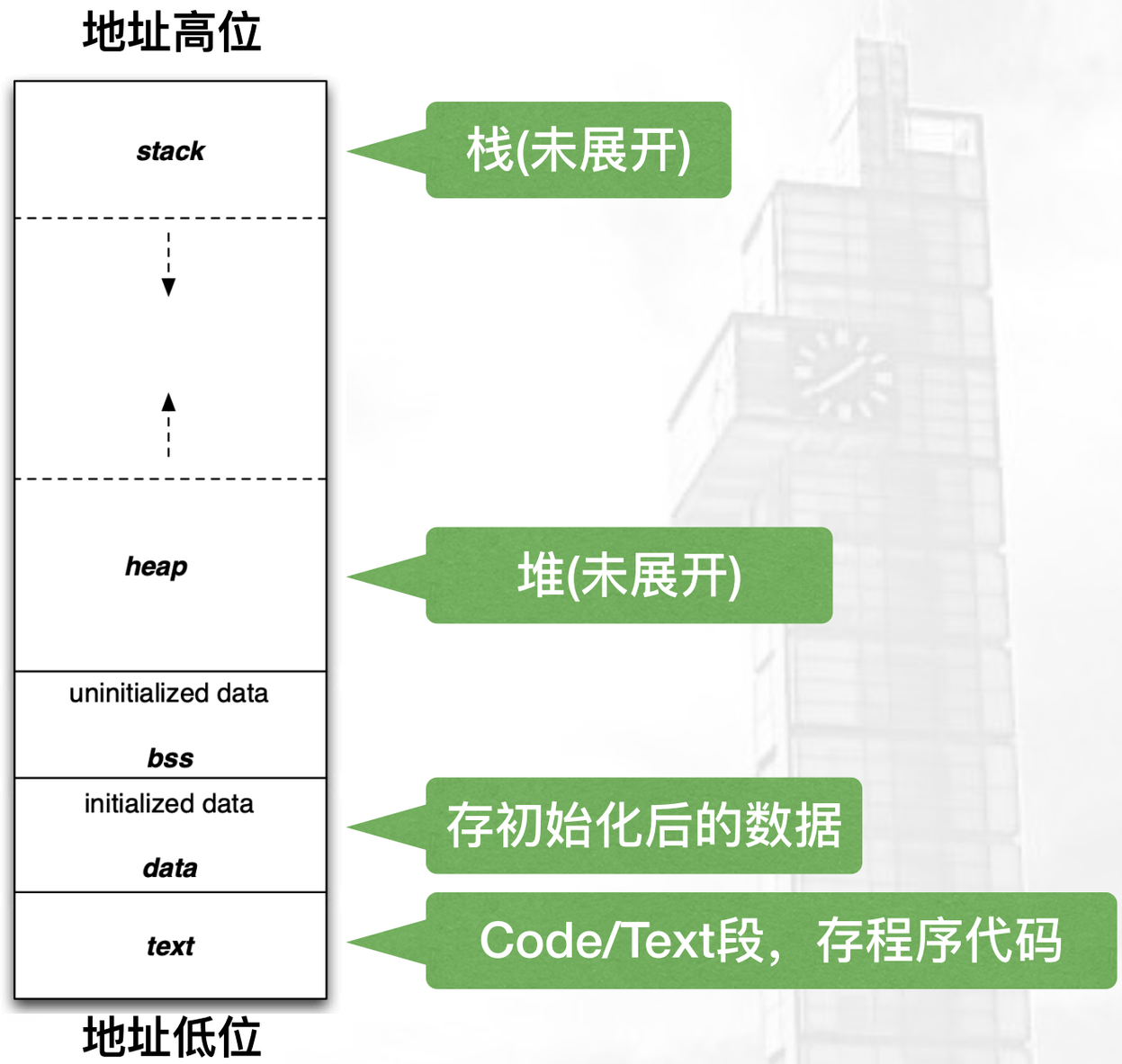
- 可执行程序的静态结构（ELF为例）
  - ELF Header
  - Sections
    - CODE/TEXT
    - DATA
    - BSS
    - HEAP
    - STACK
    - …
- Section Header Table





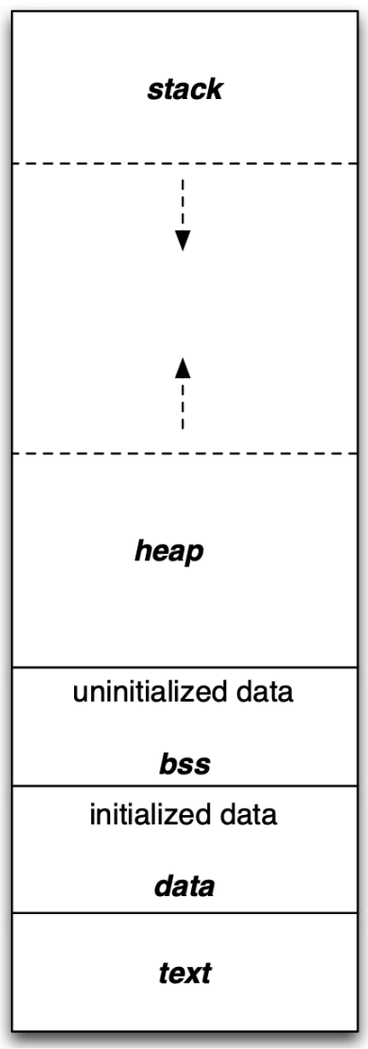
可执行程序的两个结构

- 可执行程序的静态结构（ELF为例）
  - ELF Header
  - Sections
    - CODE/TEXT
    - DATA
    - BSS
    - HEAP
    - STACK
    - …
- Section Header Table



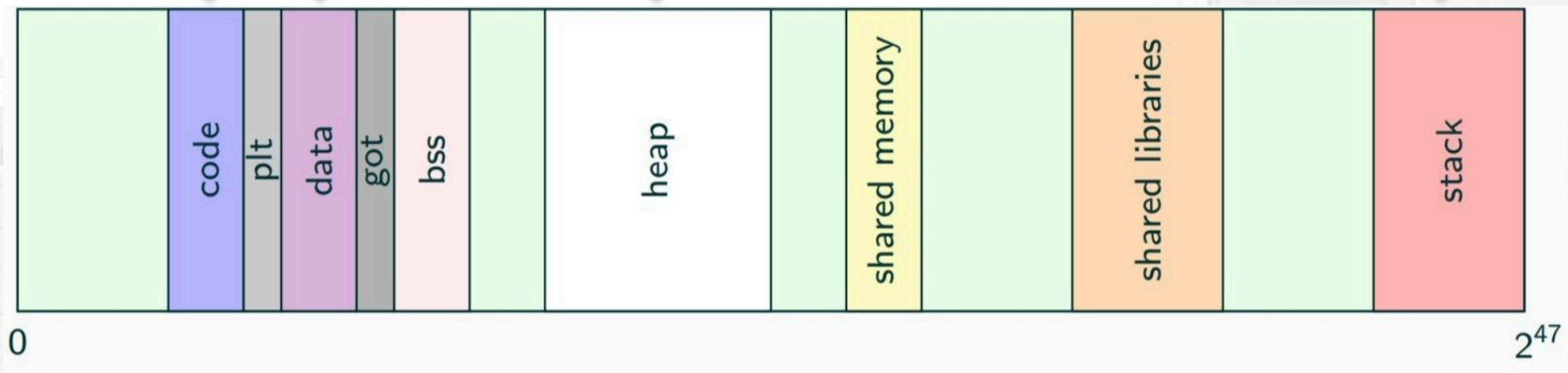
可执行程序的两个结构

地址高位



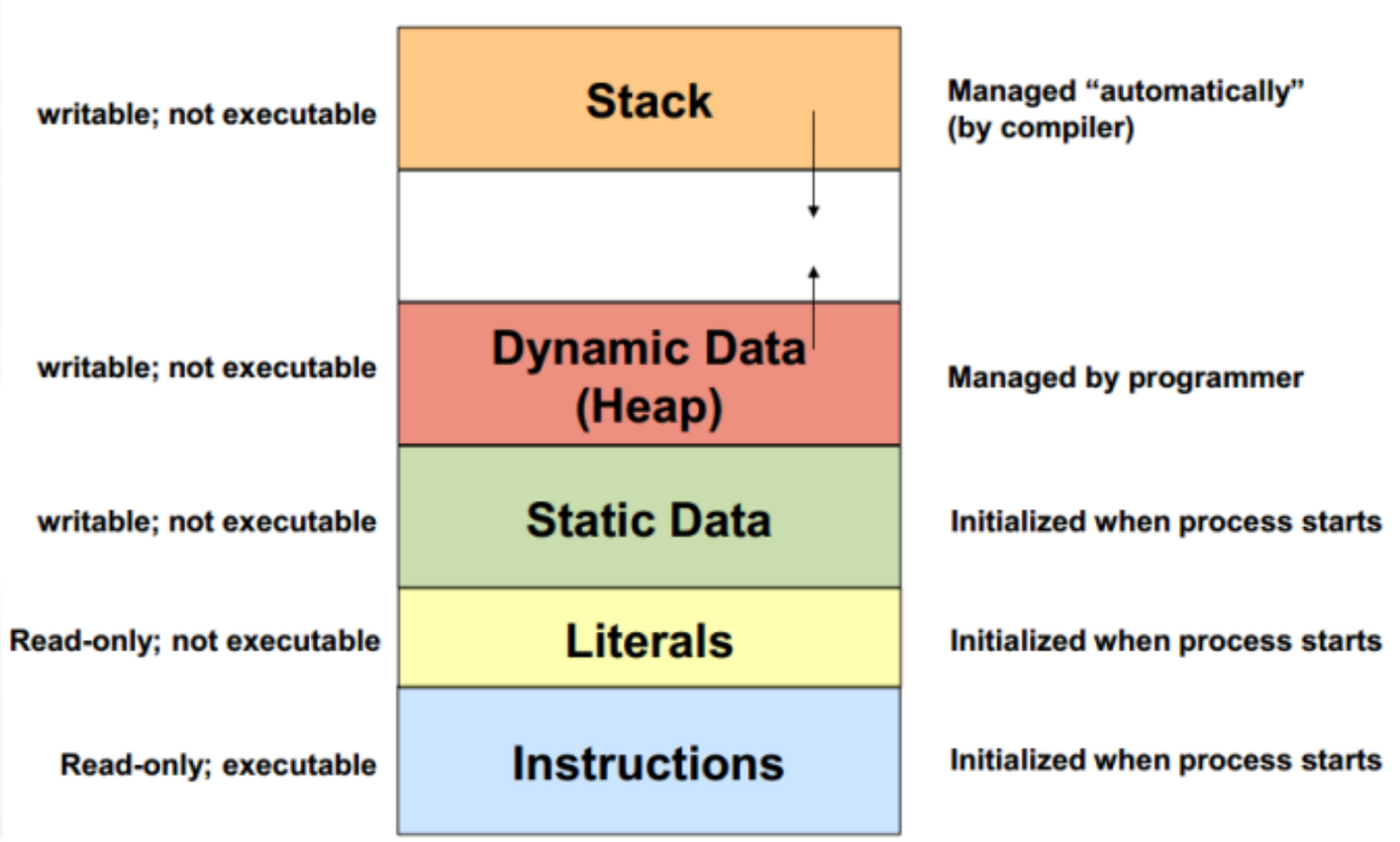
地址低位

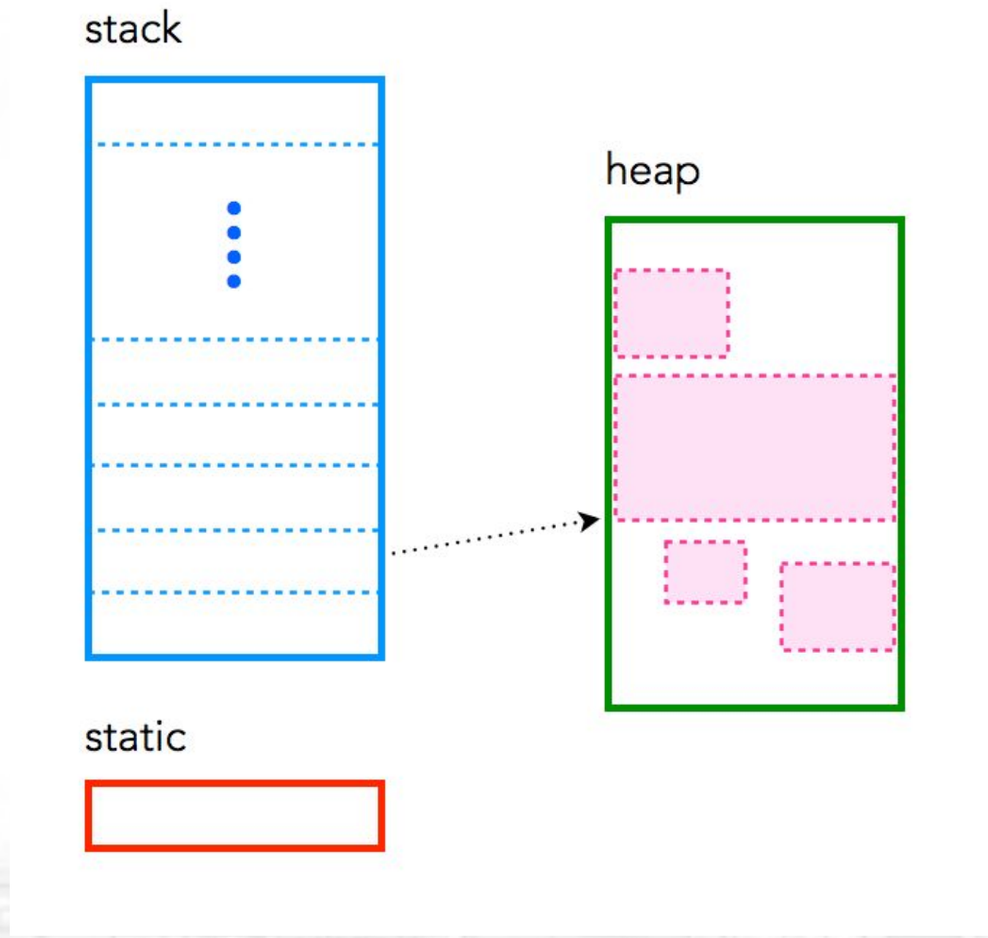
- 可执行程序的动态结构—进程!
  - 程序映射到4GB虚拟内存空间
    - 每个线程有自己的Stack/Heap
  - 虚拟地址被OS映射到实际物理地址



可执行程序的内存结构

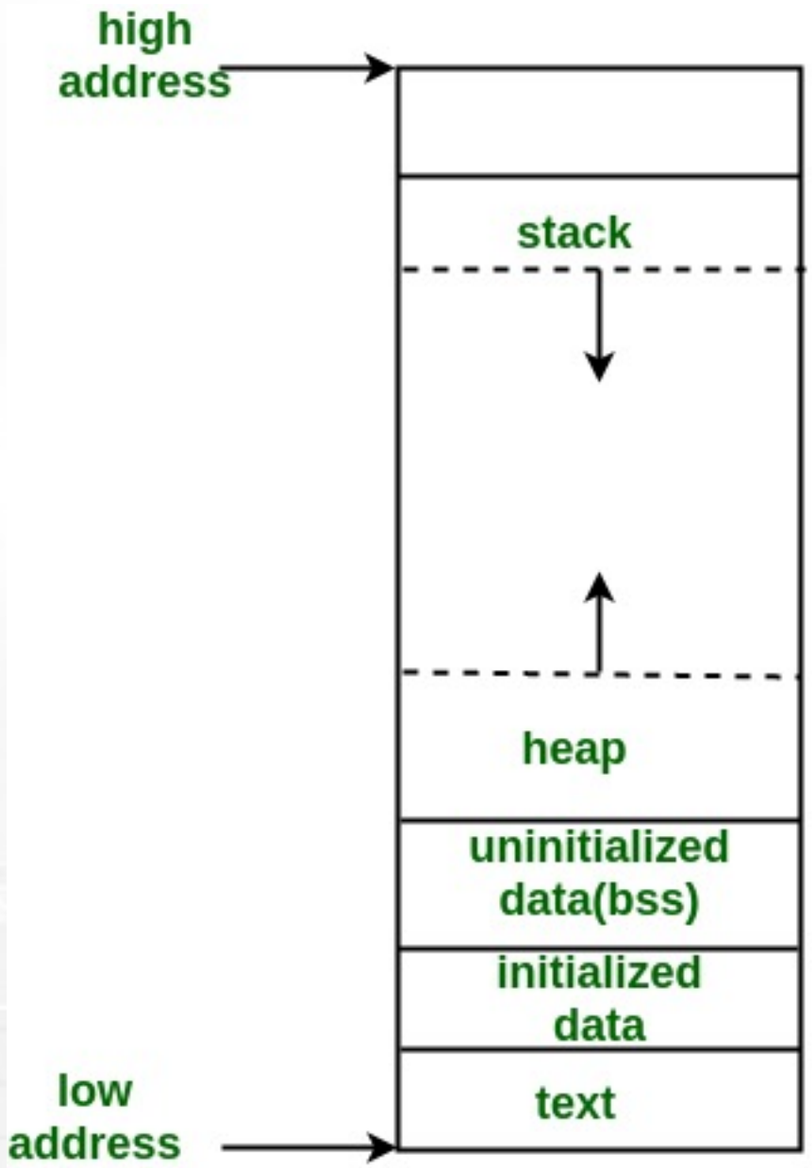
- 关于Stack/Heap



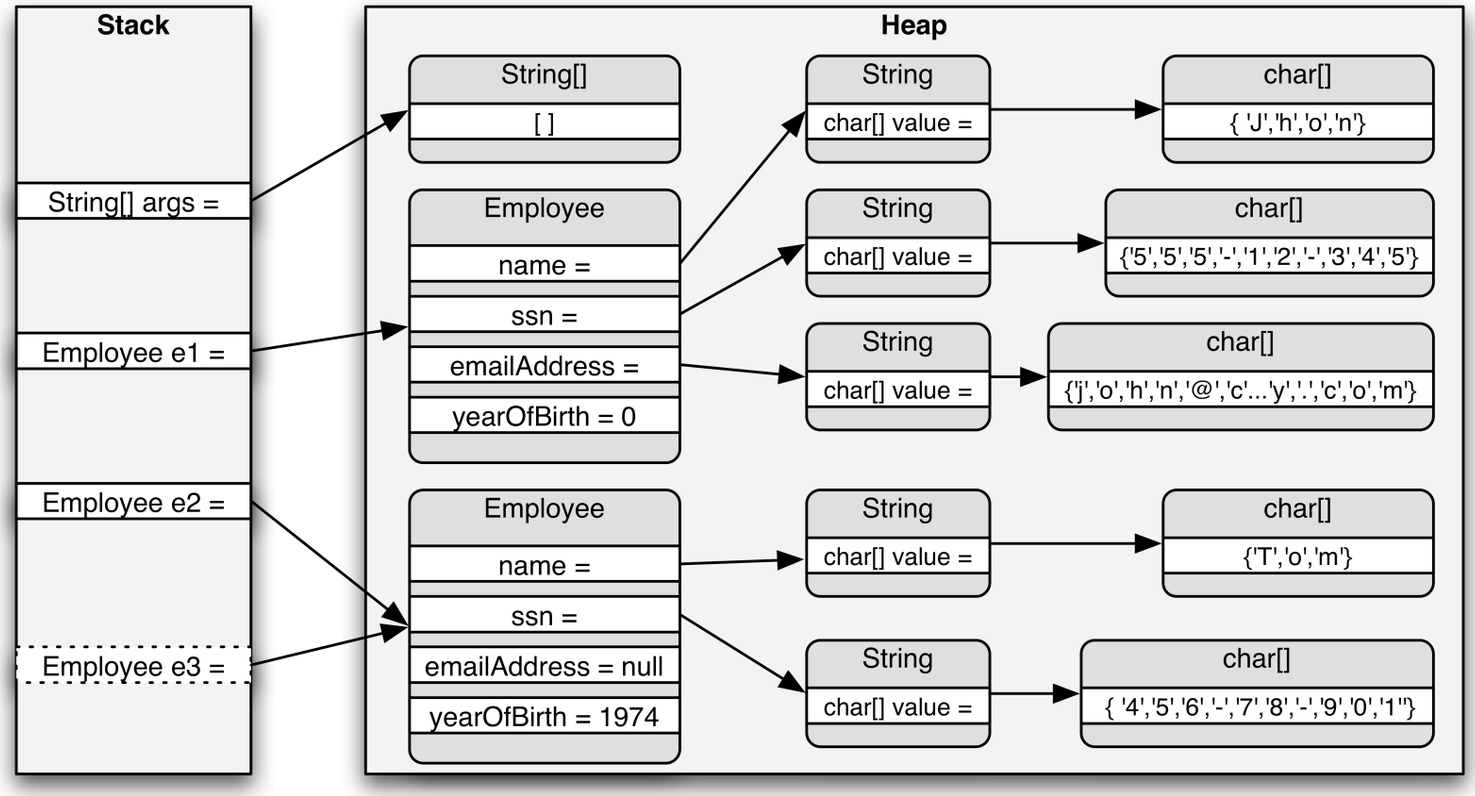


可执行程序的内存结构

- 关于Stack/Heap



一个典型的Java程序内存模型



可执行程序的内存结构

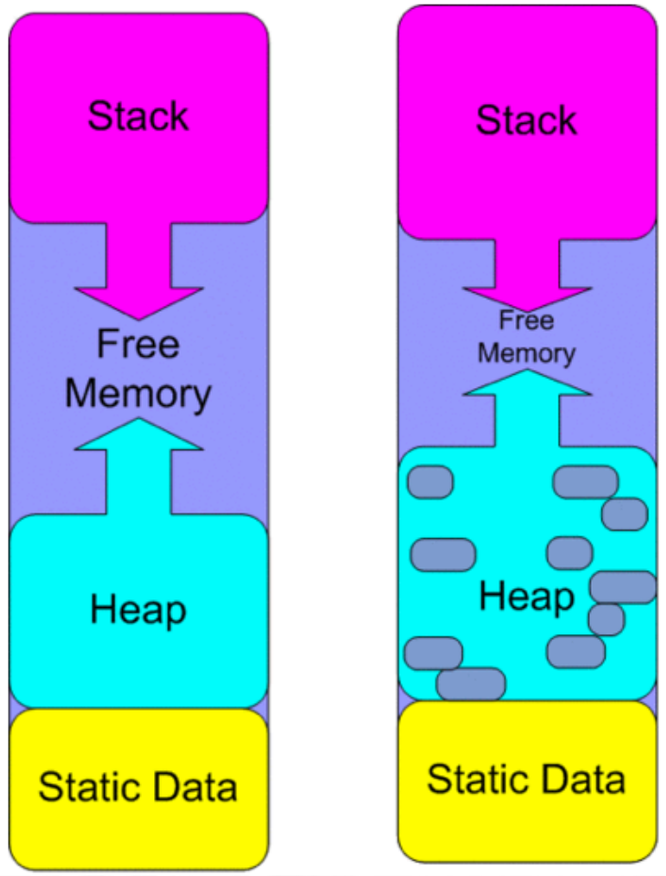
- 关于Stack/Heap
- 高性能的Heap分配器可以尽量避免碎片，显著提升性能...
- 当然对Java来说，GC的影响更大...

- **Default Allocator**

```
list<int> b;
// same as:
list< int, allocator<int> > b;
```

- **Custom Allocator**

```
#include "MyAlloc.h"
list< int, MyAlloc<int> > c;
```



So, 这和指令系统有什么关系?

CPU，是如何定位程序的?

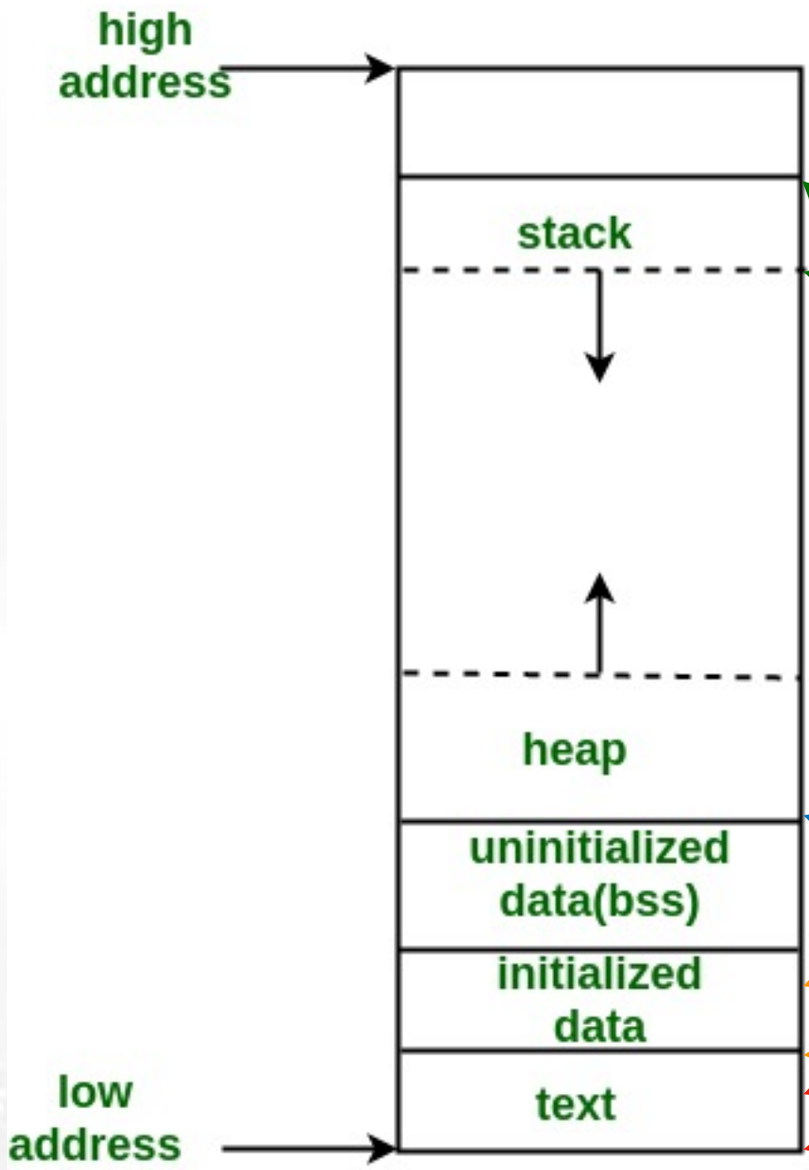
CPU的内部寄存器

8086 CPU的内部寄存器

- Registers，寄存器
  - 存储CPU运行时的各种状态，阶段，内存读写地址等
- 四类寄存器
  - 段寄存器
  - 段指针寄存器
  - 状态寄存器
  - 数据寄存器



CPU的内部寄存器



AH	AL	AX
BH	BL	BX
CH	CL	CX
DH	DL	DX

SP	堆栈指针
BP	基数指针
SI	源变址
DI	目的变址

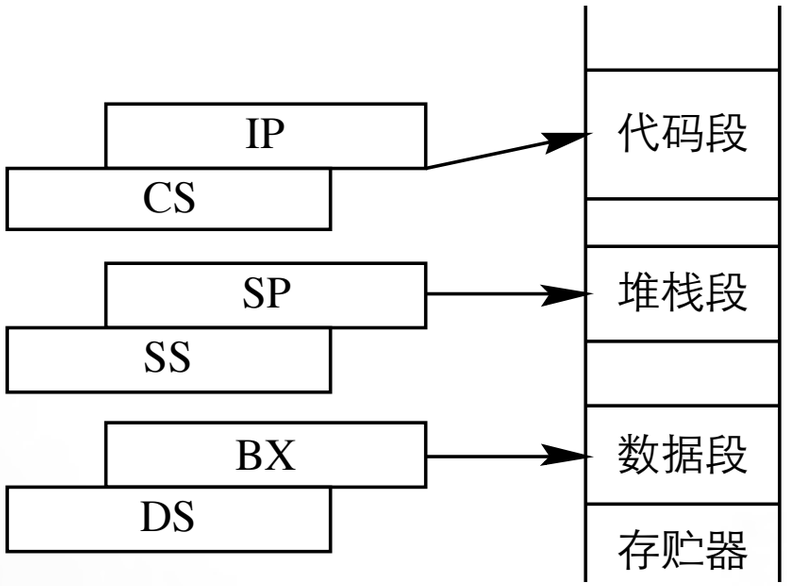
SI/DI/BX 3种

IP		指令指针
PSW H	PSW L	状态标志

CS	代码段
DS	数据段
SS	堆栈段
ES	附加段

CPU的内部寄存器

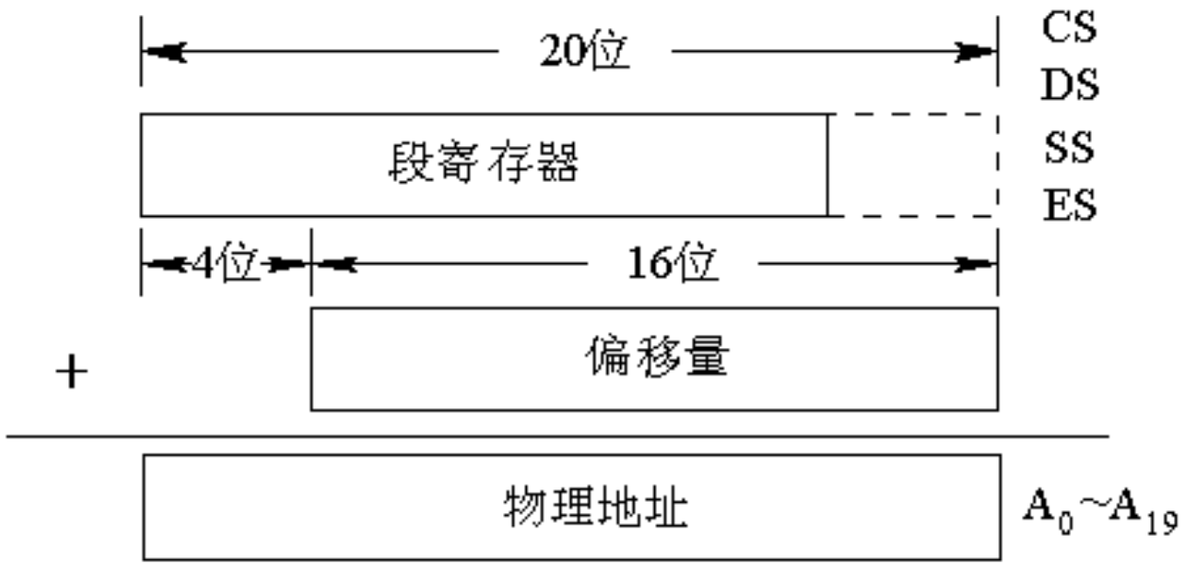
物理地址=段寄存器x16 + 偏移地址



AH	AL	AX	数据寄存器
BH	BL	BX	
CH	CL	CX	
DH	DL	DX	

SP	堆栈指针	指针寄存器
BP	基数指针	
SI	源变址	指针寄存器
DI	目的变址	
IP	指令指针	控制寄存器
PSW H / PSW L	状态标志	
CS	代码段	段寄存器
DS	数据段	
SS	堆栈段	
ES	附加段	

Then, How ???



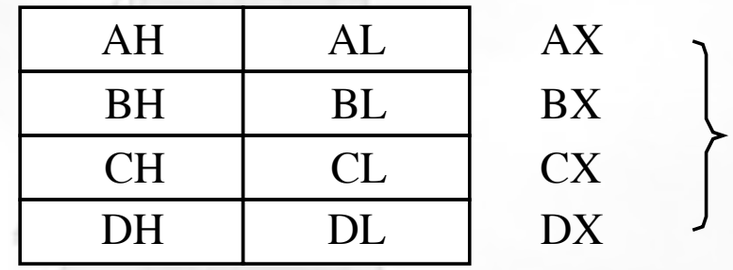
CPU的内部寄存器

- 数据寄存器:
  - 4个16位长，AX, BX, CX, DX
  - 或8个8位，AH, AL, …
- 指针寄存器: SP 和BP
  - BP是基址指针寄存器，通常用于存放基地址，以使8088的寻址更加灵活；
  - SP是堆栈指针寄存器，由它和堆栈段寄存器一起来确定堆栈内数据在内存中的位置;



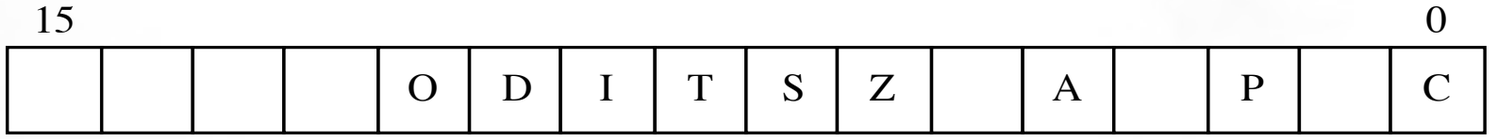
CPU的内部寄存器

- 变址寄存器: SI/DI:
  - Source/Destination Index, 源/目的变址，用于变址寻址;



CPU的内部寄存器

- 控制寄存器: PSW
  - Process State Word处理机状态字



C —进位标志位　　P —奇偶标志位
A —半加标志位　　Z — 零标志位
S —符号标志位　　T —陷阱标志位(单步标志位)
I —中断允许标志位　　D —方向标志位
O —溢出标志位

AH	AL	AX
BH	BL	BX
CH	CL	CX
DH	DL	DX

SP	堆栈指针
BP	基数指针
SI	源变址
DI	目的变址

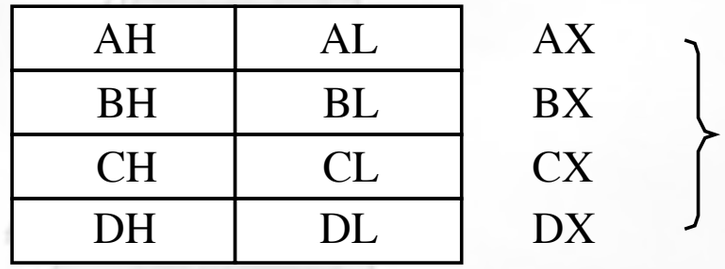
IP		指令指针
PSW H	PSW L	状态标志

CS	代码段
DS	数据段
SS	堆栈段
ES	附加段

CPU的内部寄存器

- 那么，2023年的处理呢?
  - AMD64 / x86-64

64-bit register	Lower 32 bits	Lower 16 bits	Lower 8 bits
rax	eax	ax	al
rbx	ebx	bx	bl
rcx	ecx	cx	cl
rdx	edx	dx	dl
rsi	esi	si	sil
rdi	edi	di	dil
rbp	ebp	bp	bpl
rsp	esp	sp	spl
r8	r8d	r8w	r8b
r9	r9d	r9w	r9b
r10	r10d	r10w	r10b
r11	r11d	r11w	r11b
r12	r12d	r12w	r12b
r13	r13d	r13w	r13b
r14	r14d	r14w	r14b
r15	r15d	r15w	r15b



CPU的内部寄存器

- 那么，2020年的处理呢?
  - ARM AArch64,

Registers across CPU modes

usr	sys	svc	abt	und	irq	fiq
R0						
R1						
R2						
R3						
R4						
R5						
R6						
R7						
R8						R8_fiq
R9						R9_fiq
R10						R10_fiq
R11						R11_fiq
R12						R12_fiq
R13	R13_svc		R13_abt	R13_und	R13_irq	R13_fiq
R14	R14_svc		R14_abt	R14_und	R14_irq	R14_fiq
R15						
CPSR						
	SPSR_svc		SPSR_abt	SPSR_und	SPSR_irq	SPSR_fiq

AH	AL	AX
BH	BL	BX
CH	CL	CX
DH	DL	DX

SP	堆栈指针
BP	基数指针
SI	源变址
DI	目的变址

IP		指令指针
PSW H	PSW L	状态标志

CS	代码段
DS	数据段
SS	堆栈段
ES	附加段

寻址方式

运算，
需要数据，
需要取数据，
需要知道用哪种方式去取数据，
还需要知道从哪里取数据，
—— 寻址方式
Addressing Mode !

但在此之前，
必须仔细学习一个汇编指令...

- **MOV指令**

格式：　**MOV　OPRD1，　OPRD2**

目的　　源

功能：　（**OPRD2**）→（**OPRD1**）

其中：　目的操作**OPRD1**　　源操作数 **OPRD2**

存储器　　**mem**	立即数　　**im**
通用寄存器 **r**	存储器　　**mem**
段寄存器　**SEG**（除CS）	通用寄存器 **r**
	段寄存器　**SEG**（含CS）

但也有4种例外情况：

- 目的操作数不能为立即数，或 CS、IP；

- 内存单元之间不能进行数据直接传送；

- 立即数不能直接传送到段寄存器中；

- 段寄存器之间的不能进行数据直接传送；

Done ！

寻址方式Addressing Mode

- Immediate 立即
- Direct 直接
- Register 寄存器
- Register + Indirect 寄存器间接 (2种)
- Register + Indexing 寄存器相对
- Base + Displacement 基址变址 (2种)
- Base + Displacement + Indexing 基址+变址+相对
- Implicit 隐式

寻址方式 — 立即寻址

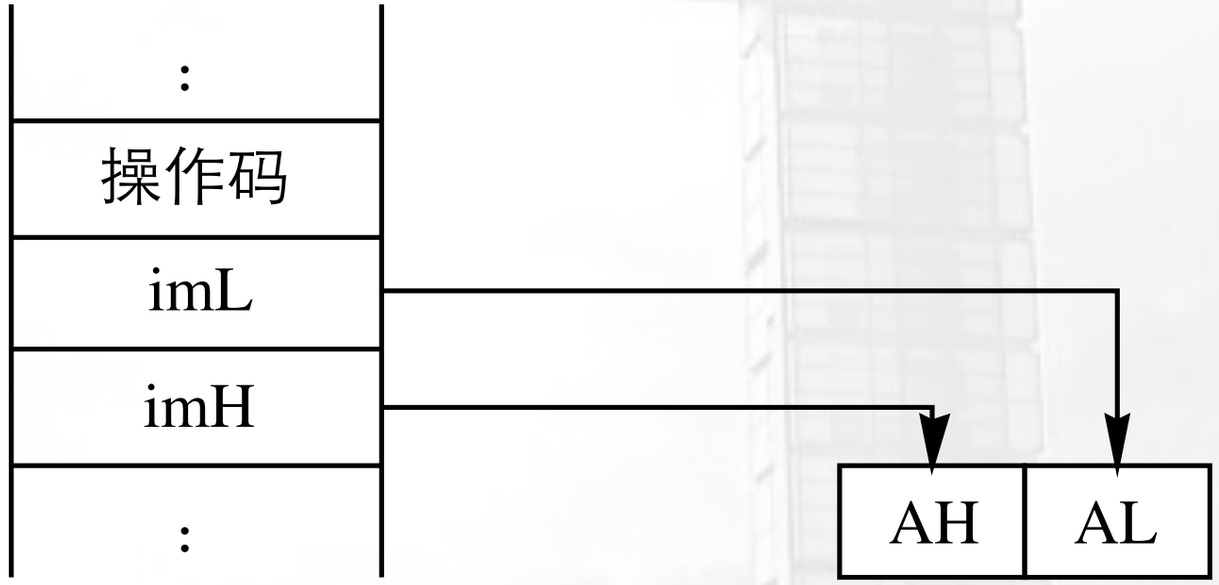
- 操作数直接包含在指令中！ (跟在操作码后面)
- 操作数是在代码段！！

例如:

MOV AL, 27H

ADD AL, 35H

MOV AX, 2000H



注意！！ 地址低位去AL，高位去AH !!!!

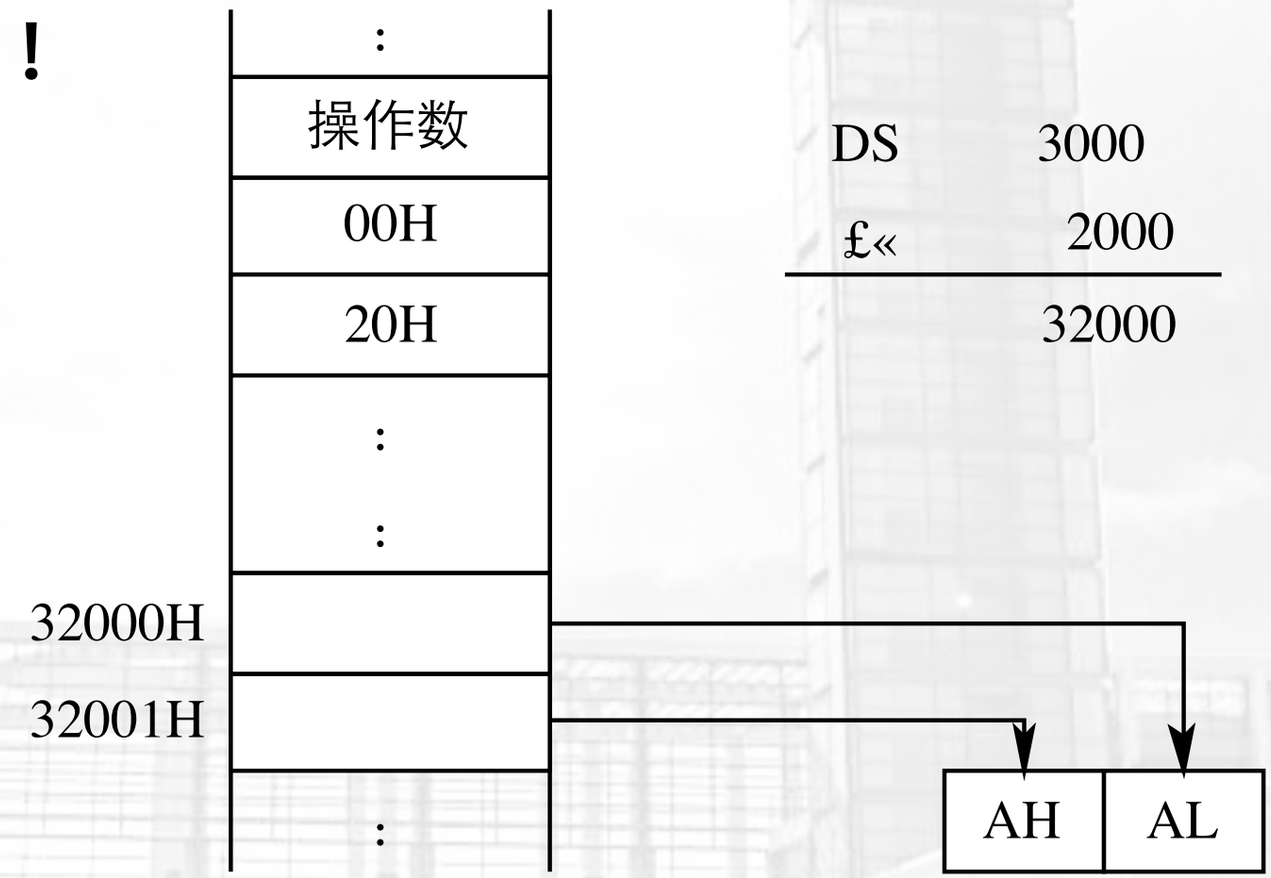
寻址方式 — 直接寻址

- **操作数的段内偏移地址直接包含在指令中！（跟在操作码后面)**
- **操作数是在数据段！！**

例如:

MOV AX, [2000H]

设DS=3000H



寻址方式 — 寄存器寻址

- 操作数在某个寄存器中!

例如:

MOV DS, AX



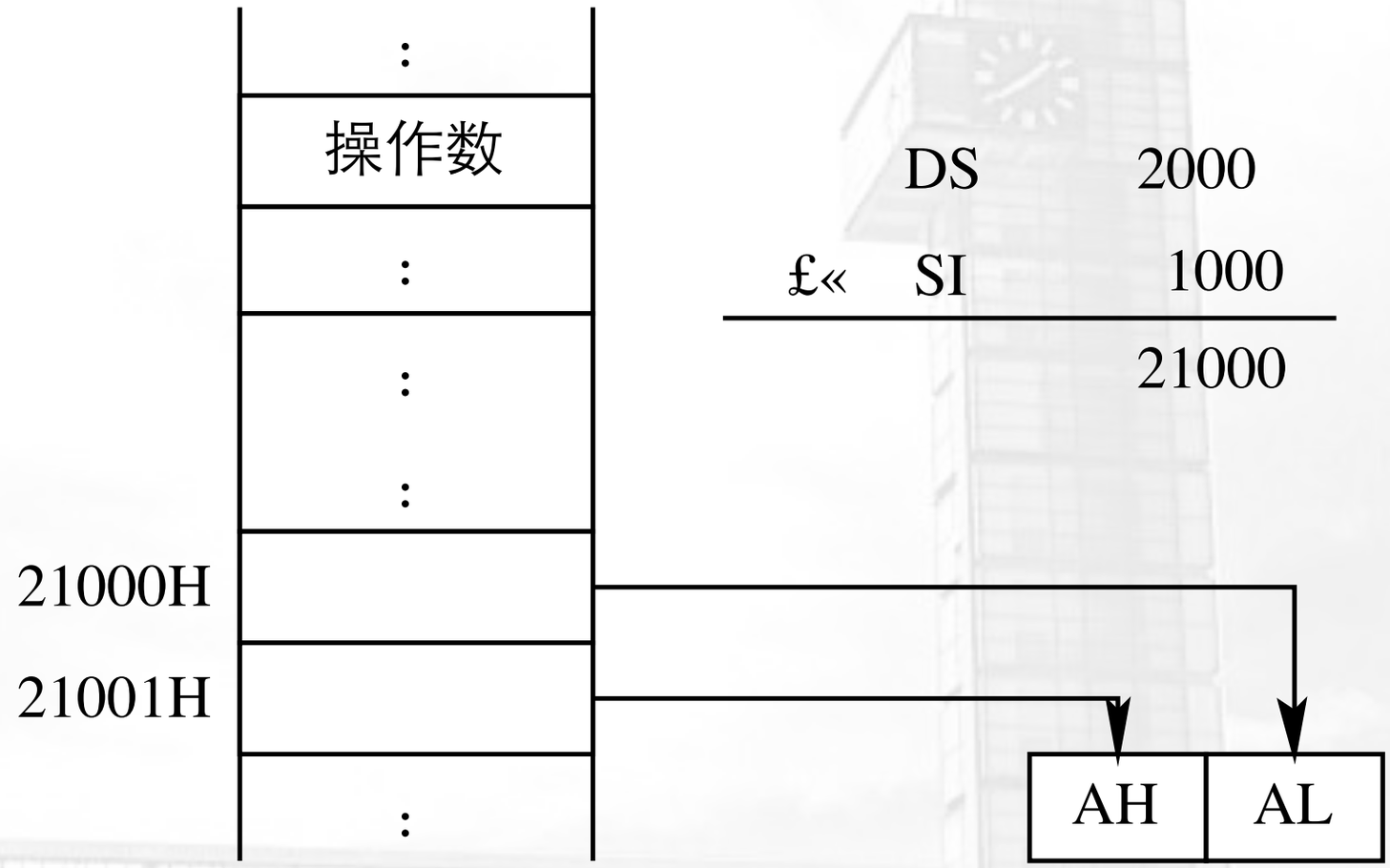
寻址方式 — 寄存器间接寻址

- 操作数的段内偏移地址放在SI/DI/BP/BX其中一个寄存器中!
- 分两种情况:
- 如果是SI/DI/BX　<—搭配DS在数据段找操作数
- 如果是BP　<—搭配SS在堆栈段找操作数

寻址方式 — 寄存器间接寻址

- 例如：

```
MOV AX,  2000H
MOV DS,   AX
MOV SI,   1000H
MOV AX,   [SI]
```

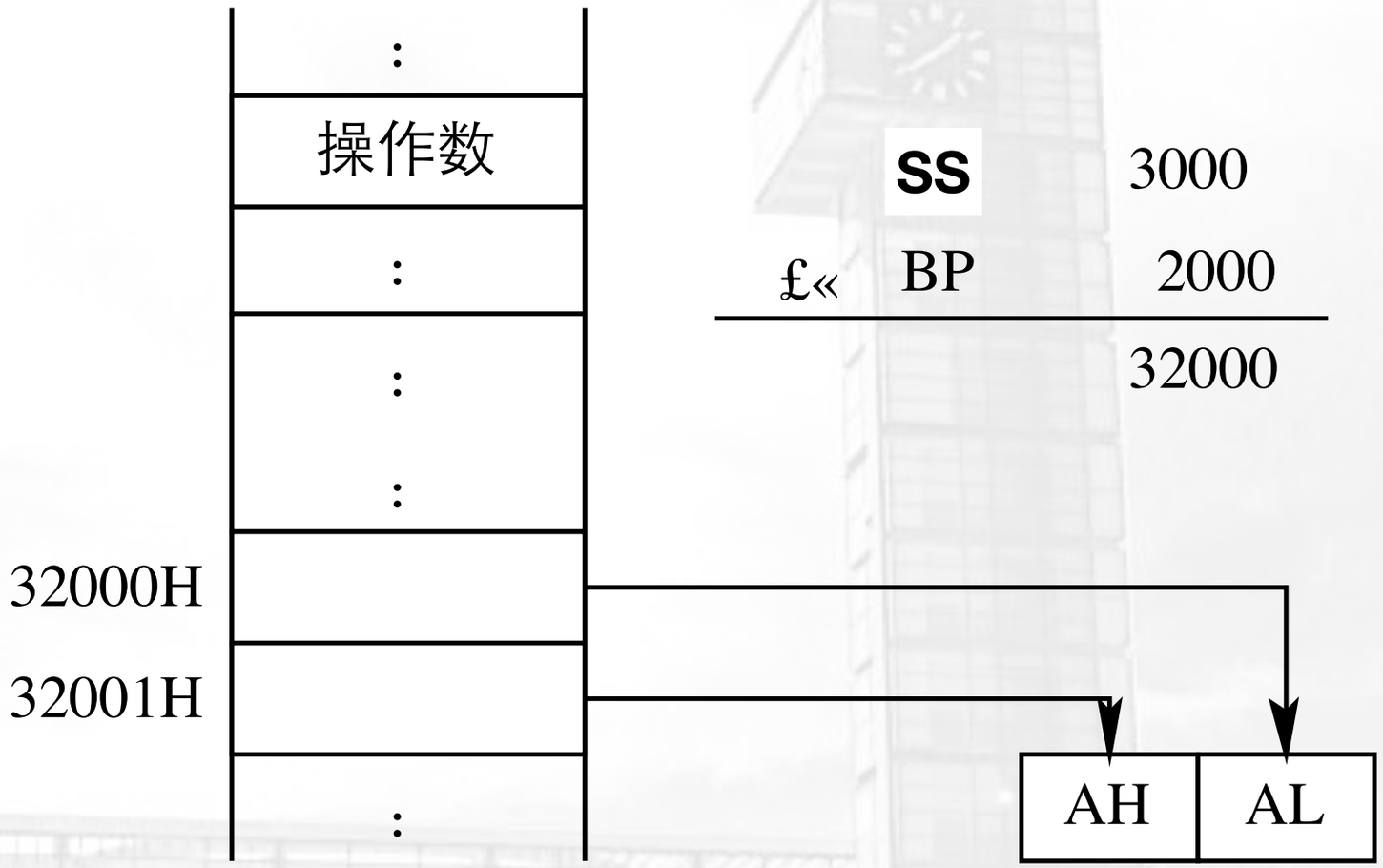


(*a*)

寻址方式 — 寄存器间接寻址

- 例如：

```
MOV AX,  3000H
MOV SS,   AX
MOV BP,    2000H
MOV AX,   [BP]
```



(*b*)

寻址方式 — 寄存器间接寻址

- 例:假设有指令：MOV BX,[DI]，在执行时，(DS)=1000H，(DI)=2345H，存储单元12345H的内容是4354H。问执行指令后，BX的值是什么?

- 解:

根据寄存器间接寻址方式的规则，在执行本例指令时，寄存器DI的值不是操作数，而是操作数的地址。该操作数的物理地址(Physical Address，PA)应由DS和DI的值形成，即:

PA=(DS)*16+DI=1000H*16+2345H=12345H。

所以，该指令的执行效果是：把从物理地址为12345H开始的一个字的值传送给BX。

寻址方式 — 寄存器间接寻址

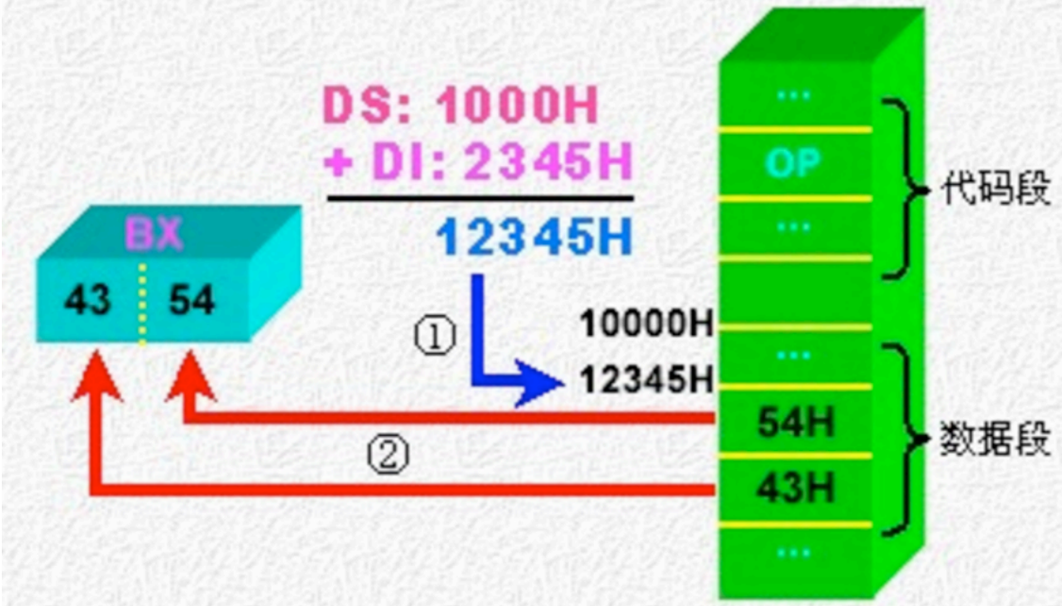
- **例:假设有指令：MOV BX,[DI]，在执行时，(DS)=1000H，(DI)=2345H，存储单元12345H的内容是4354H。问执行指令后，BX的值是什么?**

- **解:**

根据寄存器间接寻址方式的规则，在执行本例指令时，寄存器DI的值不是操作数，而是操作数的地址。该操作数的物理地址(Physical Address, PA)应由DS和DI的值形成，即:

PA=(DS)*16+DI=1000H*16+2345H=12345H。

所以，该指令的执行效果是：把从物理地址为12345H开始的一个字的值传送给BX。



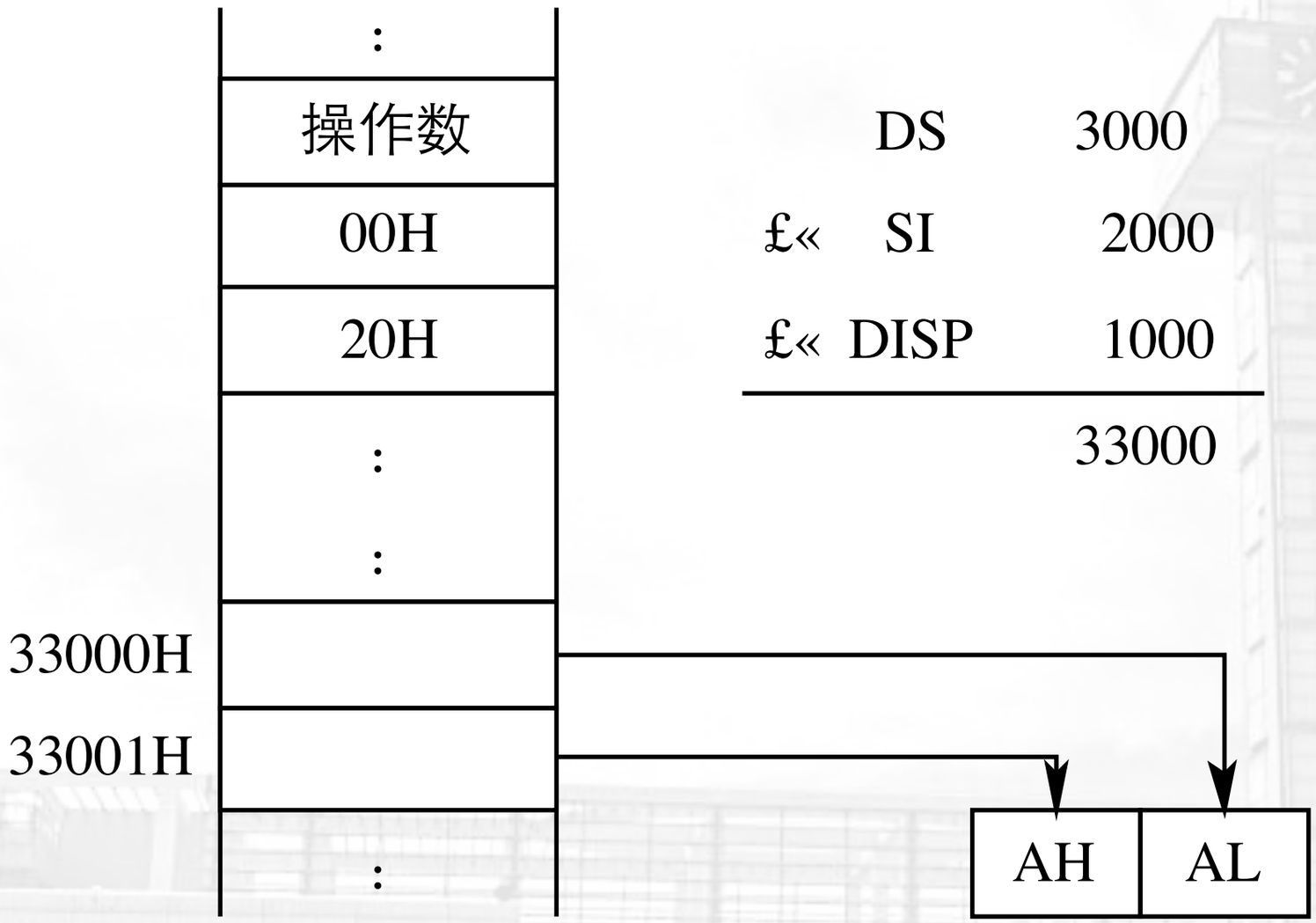
寻址方式 — 寄存器相对寻址

- 本质是寄存器间接 + 相对寻址：在寄存器间接的基础上再加上地址偏移值DISP；
- DISP跟着操作码后面，在代码段！！！
- 同样分两种情况：
  - 如果是SI/DI/BX　<—搭配DS在数据段找操作数
  - 如果是BP　<—搭配SS在堆栈段找操作数

例：MOV AX，DISP［SI］

寻址方式 — 寄存器相对寻址

例：MOV AX，DISP［SI］； DISP=1000H ; DS = 3000H; SI = 2000H



寻址方式 — 寄存器相对寻址

例: 假设指令： MOV BX, [SI+100H]，在执行它时，(DS)=1000H，(SI)=2345H，内存单元12445H的内容为2715H，问该指令执行后，BX的值是什么?

解：根据寄存器相对寻址方式的规则，在执行本例指令时，源操作数的有效地址EA为：

EA=(SI)+100H=2345H+100H=2445H

该操作数的物理地址应由DS和EA的值形成，即：

PA=(DS)*16+EA=1000H*16+2445H=12445H。

所以，该指令的执行效果是：把从物理地址为12445H开始的一个字的值传送给BX。

寻址方式 — 寄存器相对寻址

例: 假设指令：MOV BX, [SI+100H]，在执行它时，(DS)=1000H，(SI)=2345H，内存单元12445H的内容为2715H，问该指令执行后，BX的值是什么?

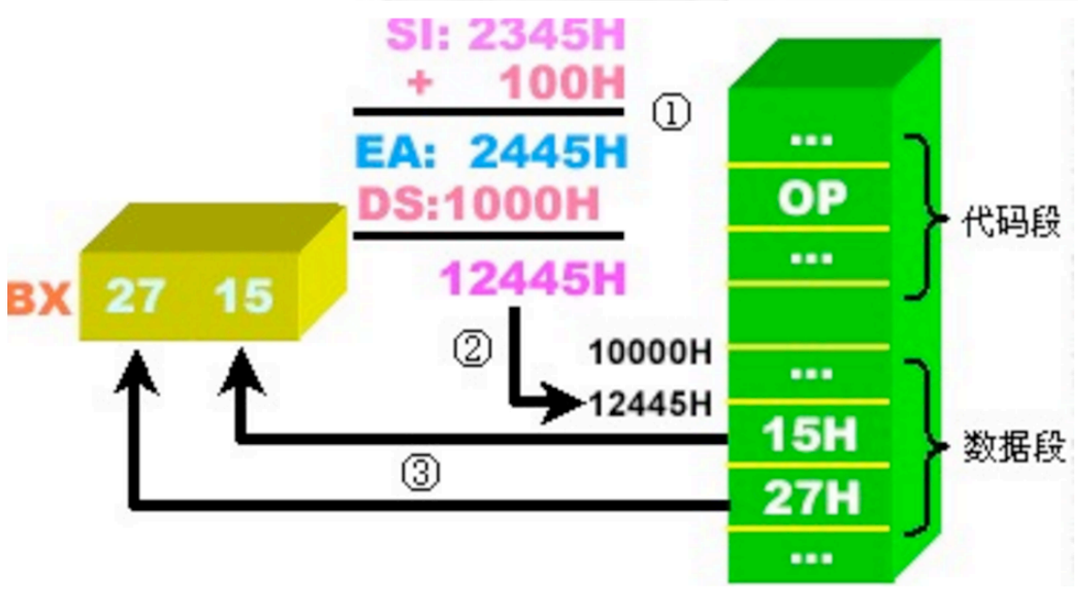
解：根据寄存器相对寻址方式的规则，在执行本例指令时，源操作数的有效地址EA为：

EA=(SI)+100H=2345H+100H=2445H

该操作数的物理地址应由DS和EA的值形成，即：

PA=(DS)*16+EA=1000H*16+2445H=12445H。

所以，该指令的执行效果是：把从物理地址为12445H开始的一个字的值传送给BX。



数据寻址方式 — 总结

- 加起来！！！ 把右边的地址值全加起来！！！
- BX找DS！！！！
- BP找SS！！！！
- “直接” v.s “间接”
- “相对” v.s “变址”

源操作数	指令的变形	源操作数的寻址方式
只有偏移量	MOV AX, [100H]	直接寻址方式
只有一个寄存器	MOV AX, [BX] 或 MOV AX, [SI]	寄存器间接寻址方式
有一个寄存器和偏移量	MOV AX, [BX+100H] 或 MOV AX, [SI+100H]	寄存器相对寻址方式
有二个寄存器	MOV AX, [BX+SI]	基址加变址寻址方式
有二个寄存器和偏移量	MOV AX, [BX+SI+100H]	相对基址加变址寻址方式